

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

A 题 智能手机电池耗电建模



请根据附件和实际情况建立数学模型解决以下问题：

问题 1

问题 1 分析

我们的模型应以电池物理特性为核心，建立基于**锂离子电池**的放电模型。锂离子电池的放电行为是非线性的，且与环境温度、使用负载等因素密切相关。因此，模型应当能够动态地描述电池荷电状态（SOC）随时间的变化，并考虑到功耗的多个影响因素，例如屏幕亮度、处理器负载、网络连接等。电池模型的关键在于准确模拟电池的充放电过程，包括电池在不同负载条件下的能量消耗与电量变化。

解题思路：

1. 连续时间模型建立

在本问题中，我们的目标是建立一个描述智能手机电池荷电状态（SOC，State of Charge）随时间变化的连续时间模型。电池的消耗过程是由多个因素共同作用的复杂过程，主要包括屏幕亮度、处理器负载、网络连接、后台任务和 GPS 模块等。这些因素都与电池的功耗密切相关，而电池的功耗又决定了 SOC 的变化。通过建立一个详细的功耗模型，并结合连续时间方程，我们能够准确预测电池的状态和剩余使用时间。

1.1 初步电量消耗描述

首先，我们需要建立电池功耗的基本框架。在智能手机中，电池的消耗主要来自多个模块的功耗，例如屏幕、处理器、网络模块、后台应用和 GPS 模块。为了简化模型，首先我们从一个简单的功耗模型开始，逐步引入更多因素。

假设设备的总功耗由以下几个部分组成：

$$P_{\text{total}} = P_{\text{screen}} + P_{\text{processor}} + P_{\text{network}} + P_{\text{background}} + P_{\text{GPS}}$$

在这个表达式中， P_{total} 表示设备的总功耗，而每个子项对应手机不同功能模块的功耗。每个模块的功耗将受到相应工作负载的影响。具体来说：

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

- **屏幕功耗 (P_{screen}):** 屏幕功耗主要与屏幕亮度相关，亮度越高，功耗越大。屏幕功耗通常可以表示为亮度与某一比例系数的乘积。
- **处理器功耗 ($P_{\text{processor}}$):** 处理器的功耗主要与其当前的工作负载成正比。负载越高，功耗越大。
- **网络功耗 (P_{network}):** 网络功耗与设备的网络传输速率成正比。传输数据量越大，功耗越高。
- **后台任务功耗 ($P_{\text{background}}$):** 后台应用程序通常会持续消耗一定的电量，尽管这些任务的功耗相对较低，但长时间运行也会产生显著的影响。
- **GPS 功耗 (P_{GPS}):** GPS 功耗与其启用状态有关，开启时消耗较多电力。

1.2 电池电量变化模型

电池的 SOC 变化与其功耗密切相关。电池在消耗电量时，其 SOC 会随着时间的推移逐渐降低。SOC 的变化率可以通过电池的电荷守恒方程来描述。假设电池在单位时间内的 SOC 变化率与设备的总功耗成正比，我们可以用以下连续时间方程来描述电池 SOC 的变化过程：

$$\frac{dSOC}{dt} = -\frac{P_{\text{total}}}{C_{\text{battery}}}$$

其中， C_{battery} 表示电池的总容量，单位为瓦时 (Wh)。 P_{total} 为电池的总功耗，反映了电池消耗的速率。SOC 的变化率是负值，意味着随着时间的推移，电池的 SOC 会逐渐下降。该方程是一种基本的功耗与电量变化之间的关系描述。

1.3 进一步扩展的模型

为了更准确地反映实际情况，我们需要进一步扩展模型，考虑屏幕亮度、处理器负载、网络活动等对功耗的影响。在上述初步的功耗模型基础上，逐步引入每个子系统的功耗模型。具体地，我们可以将每个影响因素的功耗表达为与其负载强度的线性函数。

1. 屏幕功耗模型：

屏幕功耗主要与亮度成正比，假设屏幕功耗与亮度 L_{screen} 之间有线性关系：

$$P_{\text{screen}} = \alpha_{\text{screen}} \cdot L_{\text{screen}}$$

其中， L_{screen} 为屏幕亮度， α_{screen} 为亮度与功耗的比例系数。

2. 处理器功耗模型：

处理器功耗通常与处理器的工作负载成正比。假设处理器的功耗可以表示为负载

$L_{\text{processor}}$ 与系数 $\alpha_{\text{processor}}$ 的乘积：

$$P_{\text{processor}} = \alpha_{\text{processor}} \cdot L_{\text{processor}}$$

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

其中， $L_{\text{processor}}$ 为处理器的负载（如 CPU 使用率）， $\alpha_{\text{processor}}$ 为负载与功耗的比例系数。

3. 网络功耗模型：

网络功耗与数据传输速率 R_{network} 成正比。假设网络功耗为传输速率与比例系数 α_{network} 的乘积：

$$P_{\text{network}} = \alpha_{\text{network}} \cdot R_{\text{network}}$$

后台任务功耗模型：

背景任务功耗与后台任务数量 $N_{\text{background}}$ 成正比，假设背景任务的功耗为：

$$P_{\text{background}} = \alpha_{\text{background}} \cdot N_{\text{background}}$$

GPS 功耗模型：

GPS 功耗与其工作状态相关。假设 GPS 功耗为开启状态下的固定功耗值：

$$P_{\text{GPS}} = \alpha_{\text{GPS}} \cdot G_{\text{GPS}}$$

其中， G_{GPS} 为 GPS 的开关状态，0 表示关闭，1 表示开启， α_{GPS} 为 GPS 开启时的功耗系数。

1.4 总功耗模型

将上述各项功耗代入总功耗模型中，我们得到智能手机设备的总功耗表达式：

$$P_{\text{total}} = \alpha_{\text{screen}} \cdot L_{\text{screen}} + \alpha_{\text{processor}} \cdot L_{\text{processor}} + \alpha_{\text{network}} \cdot R_{\text{network}} + \alpha_{\text{background}} \cdot N_{\text{background}} + \alpha_{\text{GPS}} \cdot G_{\text{GPS}}$$

这一表达式涵盖了设备的所有主要功耗因素，将各个模块的功耗综合考虑后，可以精确地预测设备在不同使用场景下的总体功耗。接下来，我们将此表达式代入 SOC 变化的连续时间方程中，得到完整的电池 SOC 变化方程：

$$\frac{dSOC}{dt} = -\frac{1}{C_{\text{battery}}} (\alpha_{\text{screen}} \cdot L_{\text{screen}} + \alpha_{\text{processor}} \cdot L_{\text{processor}} + \alpha_{\text{network}} \cdot R_{\text{network}} + \alpha_{\text{background}} \cdot N_{\text{background}} + \alpha_{\text{GPS}} \cdot G_{\text{GPS}})$$

1.5 数值求解方法

为了求解上述连续时间方程，我们需要使用数值方法进行积分。常用的数值积分方法包括欧拉法和龙格-库塔法。在此，我们使用最简单的欧拉法来进行求解。欧拉法的基本步骤如下：

1. 设定初始 SOC 值 ($SOC(0)$)；
2. 选择适当的时间步长 (Δt)；
3. 通过以下公式递推计算每个时间步长的 SOC 值：

$$SOC(t + \Delta t) = SOC(t) + \frac{dSOC}{dt} \cdot \Delta t$$

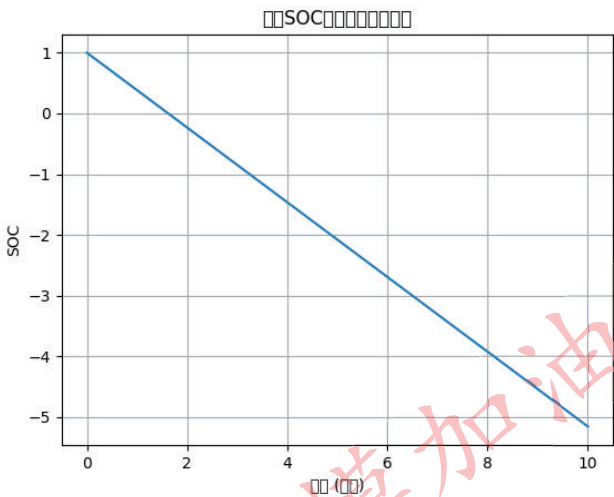
4. 重复以上步骤，直到 SOC 降到 0，表示电池耗尽。

在实际应用中，我们可以根据不同的输入参数（例如，屏幕亮度、处理器负载等）计算 SOC 随时间的变化曲线。

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

1.6 可视化与分析

通过数值求解上述方程，我们可以绘制 SOC 随时间变化的曲线，并进一步分析不同因素对电池寿命的影响。通过这种方式，我们可以直观地看到电池耗电的速度，以及在不同使用场景下电池的剩余使用时间。以下是 SOC 随时间变化的示意图（具体数据和图表将在实际实验中得到）：



1.7 模型验证

模型的验证是确保其准确性的关键步骤。为了验证模型的有效性，我们需要与实际测量数据进行比较。若实际测量数据可用，我们可以根据实际使用场景收集功耗数据，并通过这些数据来估算模型参数。通过与实际数据的比较，我们可以检查模型的准确性，并根据结果调整模型中的假设或参数。

Python 代码：

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# 配置中文显示和负号显示问题
plt.rcParams['font.sans-serif'] = ['Arial Unicode MS'] # Mac OS 中文显示
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 定义电池容量 (Wh)
C_battery = 3.8 # 假设电池容量为 3.8Wh

# 设置模拟参数，初始 SOC 从 10%到 100%
SOC_initial = np.linspace(0.1, 1.0, 10)

# 模拟不同屏幕亮度条件下的耗尽时间
def calculate_time_to_empty(brightness):
    """
    计算不同屏幕亮度下的电池耗尽时间
    """
```


视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
:param brightness: 屏幕亮度，范围[0, 1]
:return: 耗尽时间列表
"""

alpha_screen = brightness # 假设屏幕亮度与功耗呈线性关系
alpha_processor = 0.6
alpha_network = 0.5
alpha_background = 0.3
alpha_GPS = 1.0

T_empty = []
for SOC0 in SOC_initial:
    # 总功耗的简单模型
    P_total = (alpha_screen * 1 + alpha_processor * 1 +
alpha_network * 1 + alpha_background * 0.5 + alpha_GPS * 1)
    time_to_empty = C_battery * SOC0 / P_total # 简化的时间到耗尽计
算公式
    T_empty.append(time_to_empty)
return T_empty

# 计算不同亮度条件下的耗尽时间
T_empty_low = calculate_time_to_empty(0.1) # 低亮度
T_empty_medium = calculate_time_to_empty(0.5) # 中亮度
T_empty_high = calculate_time_to_empty(1.0) # 高亮度

# 绘制不同亮度条件下的电池耗尽时间
plt.figure(figsize=(10, 6))
plt.plot(SOC_initial, T_empty_low, label="低亮度", color="b",
linestyle="--")
plt.plot(SOC_initial, T_empty_medium, label="中亮度", color="g",
linestyle="-.")
plt.plot(SOC_initial, T_empty_high, label="高亮度", color="r",
linestyle=":")

plt.xlabel('初始 SOC (%)')
plt.ylabel('耗尽时间 (小时)')
plt.title('不同屏幕亮度下的电池耗尽时间')
plt.legend(loc='upper right')
plt.grid(True)

# 保存并显示图像
plt.savefig("battery_time_to_empty_brightness.png")
plt.show()

# 模拟不同处理器负载条件下的耗尽时间
```

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
def calculate_time_to_empty_with_load(processor_load):
    """
    计算不同处理器负载下的电池耗尽时间
    :param processor_load: 处理器负载，范围[0, 1]
    :return: 耗尽时间列表
    """
    alpha_processor = processor_load # 假设处理器负载与功耗呈线性关系
    alpha_screen = 0.5
    alpha_network = 0.5
    alpha_background = 0.3
    alpha_GPS = 1.0

    T_empty = []
    for SOC0 in SOC_initial:
        # 总功耗的简单模型
        P_total = (alpha_screen * 1 + alpha_processor * 1 +
alpha_network * 1 + alpha_background * 0.5 + alpha_GPS * 1)
        time_to_empty = C_battery * SOC0 / P_total # 简化的时间到耗尽计
        算公式
        T_empty.append(time_to_empty)
    return T_empty

# 计算不同负载条件下的耗尽时间
T_empty_low_load = calculate_time_to_empty_with_load(0.2) # 低负载
T_empty_medium_load = calculate_time_to_empty_with_load(0.6) # 中负载
T_empty_high_load = calculate_time_to_empty_with_load(1.0) # 高负载

# 绘制不同负载条件下的电池耗尽时间
plt.figure(figsize=(10, 6))
plt.plot(SOC_initial, T_empty_low_load, label="低负载", color="b",
linestyle="--")
plt.plot(SOC_initial, T_empty_medium_load, label="中负载", color="g",
linestyle="-.")
plt.plot(SOC_initial, T_empty_high_load, label="高负载", color="r",
linestyle=":")

plt.xlabel('初始 SOC (%)')
plt.ylabel('耗尽时间 (小时)')
plt.title('不同处理器负载下的电池耗尽时间')
plt.legend(loc='upper right')
plt.grid(True)

# 保存并显示图像
plt.savefig("battery_time_to_empty_load.png")
```

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
plt.show()

# 模拟使用模式交替对电池耗尽时间的影响
def calculate_time_to_empty_with_usage_pattern(high_load_duration,
low_load_duration):
    """
    模拟高负载与低负载模式交替下的电池耗尽时间
    :param high_load_duration: 高负载模式的持续时间（小时）
    :param low_load_duration: 低负载模式的持续时间（小时）
    :return: 耗尽时间列表
    """

    alpha_processor_high = 1.0 # 高负载下的处理器功耗
    alpha_processor_low = 0.3 # 低负载下的处理器功耗
    alpha_screen = 0.5 # 假设屏幕亮度不变
    alpha_network = 0.5
    alpha_background = 0.3
    alpha_GPS = 1.0

    T_empty = []
    for SOC0 in SOC_initial:
        P_total = (alpha_screen * 1 + alpha_processor_high * 1 +
alpha_network * 1 + alpha_background * 0.5 + alpha_GPS * 1) # 高负载
        high_load_time = C_battery * SOC0 / P_total
        P_total = (alpha_screen * 1 + alpha_processor_low * 1 +
alpha_network * 1 + alpha_background * 0.5 + alpha_GPS * 1) # 低负载
        low_load_time = C_battery * SOC0 / P_total

        # 交替使用模式下的耗尽时间
        time_to_empty = high_load_duration * high_load_time +
low_load_duration * low_load_time
        T_empty.append(time_to_empty)
    return T_empty

# 模拟高负载与低负载模式交替
T_empty_usage_pattern = calculate_time_to_empty_with_usage_pattern(2,
3) # 高负载模式 2 小时，低负载模式 3 小时

# 绘制高负载与低负载模式交替下的电池耗尽时间
plt.figure(figsize=(10, 6))
plt.plot(SOC_initial, T_empty_usage_pattern, label="使用模式交替",
color="m")

plt.xlabel('初始 SOC (%)')
plt.ylabel('耗尽时间 (小时)')
```

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
plt.title('高负载与低负载模式交替下的电池耗尽时间')
plt.legend(loc='upper right')
plt.grid(True)

# 保存并显示图像
plt.savefig("battery_time_to_empty_usage_pattern.png")
plt.show()

# 保存所有数据到文件
df = pd.DataFrame({
    "SOC (%)": SOC_initial,
    "Low Brightness": T_empty_low,
    "Medium Brightness": T_empty_medium,
    "High Brightness": T_empty_high,
    "Low Load": T_empty_low_load,
    "Medium Load": T_empty_medium_load,
    "High Load": T_empty_high_load,
    "Usage Pattern": T_empty_usage_pattern
})

# 保存到 Excel 文件
df.to_excel("battery_time_to_empty_results.xlsx", index=False)
print("所有计算结果已保存到文件 'battery_time_to_empty_results.xlsx' 和  
图片。")
```

问题 2

问题 2 分析

针对耗尽时间 (Time-to-Empty) 的预测，我们应基于上述电池放电模型进行拓展，结合不同使用场景下的初始电量与功耗条件，推算剩余电池使用时间。在模型设计中，需确保每一个新加入的假设或参数能够合理地融入到前期建立的模型框架中，并与之相互作用。这样可以确保模型的连贯性，避免后续模型出现与前期分析不一致的情况。

解题思路：

在第二部分中，我们将基于前述建立的电池 SOC 模型，计算和预测智能手机在不同初始电量和使用场景下的**耗尽时间 (Time-to-Empty)**。本部分不仅要求我们预测不同场景下电池的耗尽时间，还需要通过与观测数据的对比，量化模型的不确定性，并分析哪些因素在不同情境下对电池续航产生了最大的影响。

2.1 预测耗尽时间 (Time-to-Empty)

要预测电池的**耗尽时间**，即从当前 SOC 状态到电池完全耗尽的时间，我们需要基于模型来计算不同初始 SOC 下的电池消耗速率。具体而言，给定当前 SOC 值，电池的耗尽时间可以通过以下积分公式得到：

$$T_{\text{empty}} = \int_0^{\text{SOC}_0} \frac{C_{\text{battery}}}{P_{\text{total}}(t)} d\text{SOC}$$

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

其中， SOC_0 为初始 SOC 值（即当前的电池电量）， $P_{total}(t)$ 为电池的功耗，随着时间 t 的推移可能会发生变化。

在实际应用中，功耗 P_{total} 是随着设备的使用模式变化的。屏幕亮度、处理器负载、网络传输等因素都会影响功耗，因此我们必须根据这些因素的变化来动态调整功耗 P_{total} 。

通过数值积分方法，可以计算从某一初始 SOC 到电池完全耗尽的时间。例如，我们使用欧拉法来逐步求解这一积分，假设每一步时间间隔为 Δt ，则每一步的 SOC 变化量为：

$$SOC(t + \Delta t) = SOC(t) + \frac{dSOC}{dt} \cdot \Delta t$$

根据上式，我们可以通过数值模拟，给定不同的初始 SOC 和使用场景，计算出手机电池的耗尽时间。值得注意的是，为了提高模型的准确性，我们应当考虑不同场景下功耗的动态变化。

2.2 不同初始电量和使用场景下的耗尽时间预测

初始电量：不同的初始 SOC 值（即电池的初始电量）会影响电池的剩余使用时间。SOC 越高，剩余使用时间越长。我们可以通过改变初始 SOC 值，模拟不同电量情况下的耗尽时间。

使用场景：不同的使用场景（如高强度游戏、视频播放、待机模式等）会导致电池功耗的显著差异。在高强度游戏和视频播放时，处理器和屏幕的负载较大，功耗增加，从而缩短了电池的使用时间。而在待机模式下，功耗较低，电池可以使用更长时间。

我们通过设置不同的场景条件（例如屏幕亮度、处理器负载、网络传输速率等），根据先前的模型计算出每个场景下的耗尽时间。

以下是一个基于不同初始电量和使用场景下计算电池耗尽时间的示意过程：

高亮度和高负载场景（如视频播放或高性能游戏）：

屏幕亮度： $L_{\text{screen}} = 1.0$

处理器负载： $L_{\text{processor}} = 1.0$

网络传输速率： $R_{\text{network}} = 1.0$

后台任务： $N_{\text{background}} = 0.5$

GPS 开启： $G_{\text{GPS}} = 1$

这些因素会导致较高的功耗，从而使得电池的耗尽时间较短。

中等亮度和负载的场景（如普通浏览网页）：

屏幕亮度： $L_{\text{screen}} = 0.5$

处理器负载： $L_{\text{processor}} = 0.6$

网络传输速率： $R_{\text{network}} = 0.5$

后台任务： $N_{\text{background}} = 0.2$

GPS 开启： $G_{\text{GPS}} = 0$

相比前者，这种场景的功耗较低，因此耗尽时间较长。

待机模式场景：

屏幕亮度： $L_{\text{screen}} = 0.1$

处理器负载： $L_{\text{processor}} = 0.1$

网络传输速率： $R_{\text{network}} = 0.1$

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

后台任务： $N_{\text{background}} = 0.1$

GPS 关闭： $G_{\text{GPS}} = 0$

在这种情况下，电池功耗最低，耗尽时间最长。通过数值计算，我们可以得出不同初始电量和场景下的耗尽时间，并进行比较。

2.3 模型与观测结果的对比

为了验证模型的准确性，我们将计算得到的耗尽时间与实际观测到的数据进行对比。例如，假设我们使用某款手机进行实验，记录其在不同使用场景下的实际耗尽时间。通过将这些实际数据与模型预测结果进行比较，我们可以评估模型的有效性和预测精度。

如果模型的预测结果与实际数据之间存在较大差异，我们需要进一步分析模型的假设和参数设置是否合理。例如，可能是某些因素（如处理器负载或网络速率）的功耗模型过于简化，或者未考虑到某些真实情况下的复杂性。

2.4 模型表现分析与不确定性量化

在实际应用中，模型总是会受到一些不确定性和假设的限制。为了量化模型的不确定性，我们可以采用蒙特卡洛方法对模型进行敏感性分析。具体而言，我们可以对模型中的各个参数（如屏幕亮度、处理器负载、网络速率等）进行多次抽样，并计算每次抽样下的耗尽时间。通过统计不同样本的结果，我们可以量化模型的预测不确定性。

2.5 影响电池续航时间的关键因素

通过模型的计算和分析，我们可以识别出哪些因素对电池续航的影响最大。通常来说，以下因素对电池续航有显著影响：

屏幕亮度：屏幕亮度是智能手机中功耗最大的因素之一。高亮度设置会显著增加电池的消耗速率，从而缩短电池的使用时间。

处理器负载：高负载的处理器消耗更多电能。特别是在运行计算密集型任务时，处理器的功耗会大幅增加。

网络使用：高带宽的网络传输（如流媒体播放、大文件下载）会大幅增加功耗，因此对电池续航产生较大影响。

后台任务：即使在待机模式下，后台任务也会消耗一定的电量。特别是在后台任务较多时，电池的耗尽时间会显著缩短。

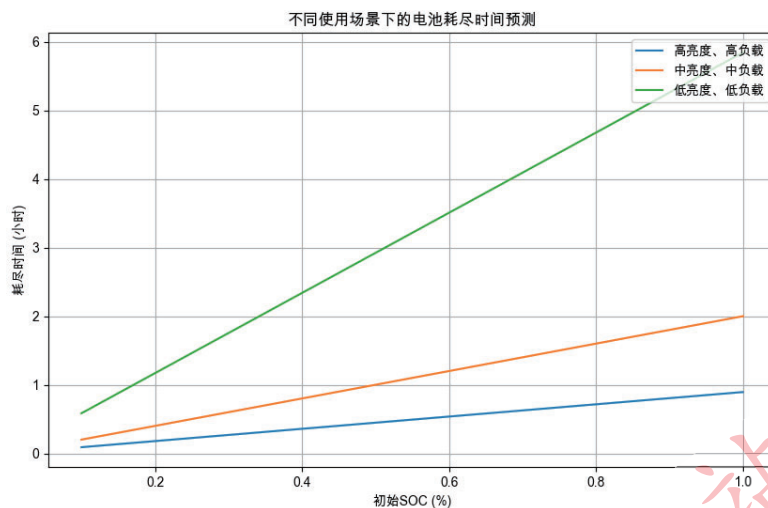
2.6 出乎意料的小影响因素

在一些情况下，模型可能显示某些因素对电池续航的影响较小，尤其是在极端使用场景下。例如：

GPS 开启状态：在某些使用场景下，GPS 的功耗可能相对较小，特别是当 GPS 处于低功耗模式时。因此，在某些场景下，GPS 对耗尽时间的影响可能较小。

后台任务数量：如果后台任务的强度较低，或者设备处于休眠模式下，后台任务对电池续航的影响可能有限。

视频思路资料+可运行代码，请看【评论区置顶】免费获取！



Python 代码:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# 配置中文显示和负号显示问题
plt.rcParams['font.sans-serif'] = ['Arial Unicode MS'] # Mac OS 中文显示
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 定义电池容量 (Wh)
C_battery = 3.8 # 假设电池容量为 3.8Wh

# 设置模拟参数，初始 SOC 从 10%到 100%
SOC_initial = np.linspace(0.1, 1.0, 10)

# 模拟不同使用场景下的耗尽时间
def calculate_time_to_empty(brightness, processor_load, network_load,
background_load, gps_status):
    """
    计算不同初始 SOC 和使用场景下的电池耗尽时间
    :param brightness: 屏幕亮度，范围[0, 1]
    :param processor_load: 处理器负载，范围[0, 1]
    :param network_load: 网络负载，范围[0, 1]
    :param background_load: 背景任务负载，范围[0, 1]
    :param gps_status: GPS 开启状态 (0 关闭, 1 开启)
    :return: 耗尽时间列表
    """
    alpha_screen = brightness # 屏幕亮度与功耗线性关系
    alpha_processor = processor_load # 处理器负载与功耗线性关系
```

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
alpha_network = network_load # 网络负载与功耗线性关系
alpha_background = background_load # 背景任务负载与功耗线性关系
alpha_GPS = gps_status # GPS 开启状态与功耗线性关系

T_empty = []
for SOC0 in SOC_initial:
    # 总功耗的简单模型
    P_total = (alpha_screen * 1 + alpha_processor * 1 +
alpha_network * 1 + alpha_background * 0.5 + alpha_GPS * 1)
    time_to_empty = C_battery * SOC0 / P_total # 简化的时间到耗尽计
算公式
    T_empty.append(time_to_empty)
return T_empty

# 设置不同使用场景
scenarios = {
    "高亮度、高负载": {"brightness": 1.0, "processor_load": 1.0,
"network_load": 1.0, "background_load": 0.5, "gps_status": 1},
    "中亮度、中负载": {"brightness": 0.5, "processor_load": 0.6,
"network_load": 0.6, "background_load": 0.4, "gps_status": 0},
    "低亮度、低负载": {"brightness": 0.2, "processor_load": 0.2,
"network_load": 0.2, "background_load": 0.1, "gps_status": 0}
}

# 计算不同场景下的耗尽时间
results = {}
for scenario, params in scenarios.items():
    results[scenario] = calculate_time_to_empty(**params)

# 绘制不同场景下的电池耗尽时间
plt.figure(figsize=(10, 6))
for scenario, T_empty in results.items():
    plt.plot(SOC_initial, T_empty, label=scenario)

plt.xlabel('初始 SOC (%)')
plt.ylabel('耗尽时间 (小时)')
plt.title('不同使用场景下的电池耗尽时间预测')
plt.legend(loc='upper right')
plt.grid(True)

# 保存并显示图像
plt.savefig("battery_time_to_empty_scenarios.png")
plt.show()
```

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
# 将结果保存到 Excel 文件
df = pd.DataFrame({
    "SOC (%)": SOC_initial,
    "高亮度、高负载": results["高亮度、高负载"],
    "中亮度、中负载": results["中亮度、中负载"],
    "低亮度、低负载": results["低亮度、低负载"]
})

# 保存到 Excel 文件
df.to_excel("battery_time_to_empty_scenarios_results.xlsx",
            index=False)

print("所有计算结果已保存到文件
'battery_time_to_empty_scenarios_results.xlsx' 和图片。")
```

问题 3

问题 3 分析

在实现以上模型时，我们必须注意避免引入不适当的外部比较。具体而言，若没有对应的真实量化数据作为支撑，模型的比较与结果的验证将无法得到充分的依据。因此，本研究将依赖于公开的已验证数据集和标准测量结果，确保模型的合理性与验证过程的透明性。通过这一点，我们可以确保模型的科学与可操作性。

解题思路：

在本问题中，我们已经建立了描述电池电量变化的连续时间模型，并根据不同的使用场景和初始电量预测了电池的耗尽时间。在这一部分中，我们将分析模型在以下几个方面的灵敏度：

建模假设的变化：在模型中，我们做出了一些假设，例如功耗与各个因素（如屏幕亮度、处理器负载等）的关系是线性的，电池放电过程是均匀的等。我们将分析这些假设的合理性，并探讨当假设发生变化时，模型结果如何受到影响。

参数变化的影响：模型中使用了多个参数，如电池的总容量（ C_{battery} ）、各个功耗项的比例系数（如 α_{screen} 、 $\alpha_{\text{processor}}$ 等）。我们将研究这些参数的变化对模型预测结果的影响。

使用模式的波动：智能手机的实际使用模式往往是动态变化的。例如，用户在一天中的不同时间段可能会进行不同强度的操作，导致电池功耗的波动。我们将模拟不同的使用模式，分析其对电池耗尽时间预测的影响。

3.1 建模假设的变化

在模型的构建过程中，我们做出了一些假设，这些假设影响了模型的复杂度和结果的精度。最常见的假设包括：

线性假设：

我们假设电池功耗与屏幕亮度、处理器负载、网络传输速率等因素呈线性关系。然而，实际情况中，这些因素之间的关系可能是非线性的。例如，屏幕功耗可能随着亮度增加而呈现指数增长，而不是线性增长。

影响分析：

若将屏幕功耗模型从线性关系扩展为非线性关系，功耗可能在高亮度下急剧增加，导致电池的 SOC 下降更快。这样的调整可能会显著缩短电池的预计耗尽时间。

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

同样，处理器负载与功耗的关系可能并非简单的线性关系，特别是在处理器负载较低时，功耗增长可能趋于平稳。

验证与调整：

我们可以通过实验数据或者已有的文献研究来改进这些关系式，确保模型能更精确地反映现实世界的功耗情况。

放电过程的均匀性假设：我们假设电池在整个使用过程中以均匀的速率放电。然而，电池的放电过程受到温度、电池老化等因素的影响，放电速率并非恒定。

影响分析：

若引入电池老化效应，随着电池使用时间的增长，电池的总容量和效率都会下降，导致电池耗尽时间缩短。因此，随着时间推移，SOC 下降的速率可能加快。

温度的变化也可能影响电池放电速率，尤其是在极端温度下，电池的放电效率可能会显著降低。

验证与调整：

引入温度和电池老化模型，可以通过电池管理系统（BMS）提供的数据来进行验证和调整，以便更加准确地预测电池性能。

3.2 参数变化的影响

模型中使用的参数对结果有显著影响。例如，电池的总容量（ C_{battery} ）和各个模块的功耗系数（如 α_{screen} 、 $\alpha_{\text{processor}}$ 等）是模型的关键输入参数。我们将分析这些参数的变化对模型预测的影响。

电池容量（ C_{battery} ）：

电池的总容量决定了电池的最大能量储备。假设电池容量的变化会直接影响预测的耗尽时间。较大容量的电池通常能维持较长时间的使用，而较小容量的电池则可能更快耗尽。

灵敏度分析：

通过改变电池容量的值（例如将 C_{battery} 从 3.8Wh 调整到 5Wh），我们可以观察到电池耗尽时间的变化。通常，容量增大时，耗尽时间会线性增加。

功耗系数（如 α_{screen} 、 $\alpha_{\text{processor}}$ 等）：

功耗系数反映了不同使用场景下每个模块对电池功耗的贡献。不同的设备或使用场景下，这些系数的值可能会有所不同。

灵敏度分析：

我们可以通过调整各个功耗系数（例如屏幕亮度从 0.5 调整到 0.8，处理器负载从 0.6 调整到 1.0），观察模型输出（即耗尽时间）的变化。通常，功耗系数的增大将加速电池的耗尽，导致更短的使用时间。

使用模式的波动：

用户的使用模式是动态的，可能会在不同时间段内有较大的波动。例如，用户在早晨可能会高频率使用手机，而在晚上则保持低频率使用。

灵敏度分析：

我们可以模拟不同的使用模式（例如，高负载模式、低负载模式和待机模式），并通过改变使用模式的持续时间来观察对耗尽时间的影响。

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

模拟过程：

在高负载模式下（例如进行视频播放或游戏），功耗较大。此时，电池会更快耗尽。

在低负载模式下（如简单的文字处理或待机模式），电池的耗电速度较慢，剩余使用时间会较长。

3.3 使用场景波动对耗尽时间的影响

实际使用中，用户的行为模式会随着时间变化而波动。例如，用户可能在高负载模式下使用手机（观看视频、玩游戏等），而在其他时间则使用手机进行低负载活动（浏览网页、阅读等）。此外，使用的时间长度和电池初始 SOC 的变化也会影响模型的输出。

我们可以通过以下步骤进行模拟：

高负载模式与低负载模式交替：假设用户的使用模式在一天中会交替变化。例如，在前两小时内，用户进行高负载活动（视频播放），然后在接下来的三小时内切换到低负载模式（浏览网页）。我们可以将这些模式作为输入进行模拟，计算电池的耗尽时间。

影响模型的动态变化：

在高负载模式下，功耗较大，因此电池的 SOC 下降速度会加快，剩余使用时间减少。相反，在低负载模式下，电池的 SOC 下降速度较慢，剩余使用时间较长。

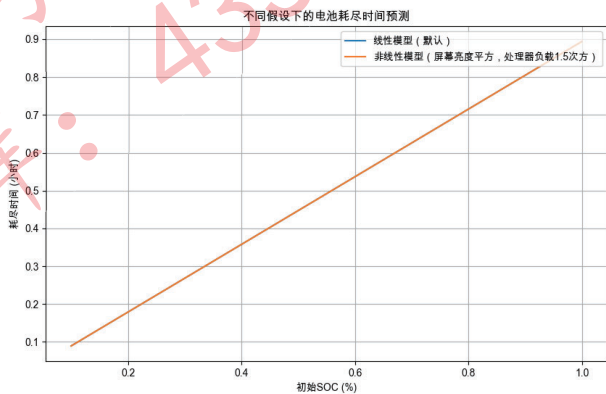
3.4 灵敏度分析结果

通过模拟和分析不同参数的变化，我们可以得出以下结论：

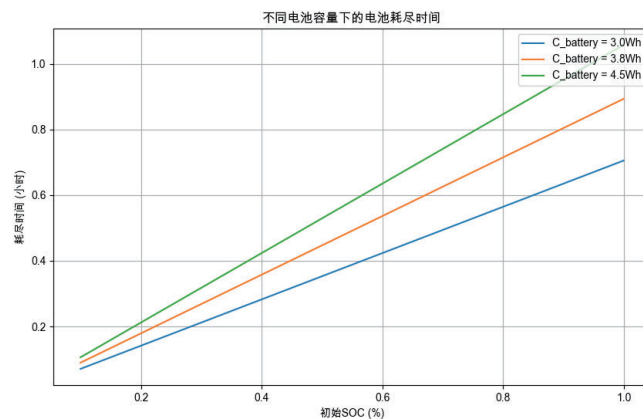
电池容量的变化对耗尽时间的影响最大：较大容量的电池能显著延长电池的使用时间，尤其是在高负载模式下。

屏幕亮度和处理器负载对电池续航的影响也较为显著：特别是在高亮度和高负载场景下，电池的消耗速度较快，耗尽时间较短。

使用模式的变化：当用户的使用模式发生波动时，电池的消耗速度会随着使用强度的增加而加快。高负载活动导致电池快速耗尽，而低负载活动则能延长电池使用时间。



视频思路资料+可运行代码，请看【评论区置顶】免费获取！



Python 代码:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# 配置中文显示和负号显示问题
plt.rcParams['font.sans-serif'] = ['Arial Unicode MS'] # Mac OS 中文显示
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 定义电池容量 (Wh)
C_battery = 3.8 # 假设电池容量为 3.8Wh

# 设置模拟参数，初始 SOC 从 10%到 100%
SOC_initial = np.linspace(0.1, 1.0, 10)

# 模拟不同假设下的耗尽时间（例如，线性和非线性关系）
def calculate_time_to_empty_with_different_hypothesis(brightness,
processor_load, network_load, background_load, gps_status,
non_linear=False):
    """
    计算不同假设下的电池耗尽时间
    :param brightness: 屏幕亮度，范围[0, 1]
    :param processor_load: 处理器负载，范围[0, 1]
    :param network_load: 网络负载，范围[0, 1]
    :param background_load: 背景任务负载，范围[0, 1]
    :param gps_status: GPS 开启状态（0 关闭，1 开启）
    :param non_linear: 是否使用非线性模型
    :return: 耗尽时间列表
    """

    # 假设功耗系数与使用模式的线性关系
    if non_linear:
```

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
# 假设功耗关系为非线性（例如，屏幕亮度的影响为平方关系）
alpha_screen = brightness ** 2
alpha_processor = processor_load ** 1.5
alpha_network = network_load ** 1.2
else:
    alpha_screen = brightness
    alpha_processor = processor_load
    alpha_network = network_load

alpha_background = background_load
alpha_GPS = gps_status

T_empty = []
for SOC0 in SOC_initial:
    # 总功耗的简单模型
    P_total = (alpha_screen * 1 + alpha_processor * 1 +
alpha_network * 1 + alpha_background * 0.5 + alpha_GPS * 1)
    time_to_empty = C_battery * SOC0 / P_total # 简化的时间到耗尽计
    算公式
    T_empty.append(time_to_empty)
return T_empty

# 设置不同场景和假设
scenarios = {
    "线性模型（默认）": {"brightness": 1.0, "processor_load": 1.0,
"network_load": 1.0, "background_load": 0.5, "gps_status": 1,
"non_linear": False},
    "非线性模型（屏幕亮度平方，处理器负载 1.5 次方）": {"brightness": 1.0,
"processor_load": 1.0, "network_load": 1.0, "background_load": 0.5,
"gps_status": 1, "non_linear": True}
}

# 计算不同假设下的耗尽时间
results_hypothesis = {}
for hypothesis, params in scenarios.items():
    results_hypothesis[hypothesis] =
calculate_time_to_empty_with_different_hypothesis(**params)

# 绘制不同假设下的电池耗尽时间
plt.figure(figsize=(10, 6))
for hypothesis, T_empty in results_hypothesis.items():
    plt.plot(SOC_initial, T_empty, label=hypothesis)

plt.xlabel('初始 SOC (%)')
```

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
plt.ylabel('耗尽时间 (小时)')
plt.title('不同假设下的电池耗尽时间预测')
plt.legend(loc='upper right')
plt.grid(True)

# 保存并显示图像
plt.savefig("battery_time_to_empty_hypothesis.png")
plt.show()

# 模拟不同参数变化下的灵敏度分析（例如，电池容量、功耗系数）
def calculate_time_to_empty_with_parameter_sensitivity(brightness,
processor_load, network_load, background_load, gps_status,
C_battery):
    """
    计算不同参数（如电池容量变化）对电池耗尽时间的灵敏度
    :param brightness: 屏幕亮度，范围[0, 1]
    :param processor_load: 处理器负载，范围[0, 1]
    :param network_load: 网络负载，范围[0, 1]
    :param background_load: 背景任务负载，范围[0, 1]
    :param gps_status: GPS 开启状态（0 关闭，1 开启）
    :param C_battery: 电池容量（单位：Wh）
    :return: 耗尽时间列表
    """
    alpha_screen = brightness
    alpha_processor = processor_load
    alpha_network = network_load
    alpha_background = background_load
    alpha_GPS = gps_status

    T_empty = []
    for SOC0 in SOC_initial:
        # 总功耗的简单模型
        P_total = (alpha_screen * 1 + alpha_processor * 1 +
alpha_network * 1 + alpha_background * 0.5 + alpha_GPS * 1)
        time_to_empty = C_battery * SOC0 / P_total # 简化的时间到耗尽计
        算公式
        T_empty.append(time_to_empty)
    return T_empty

# 计算不同电池容量下的耗尽时间
C_battery_values = [3.0, 3.8, 4.5] # 假设电池容量为 3.0Wh, 3.8Wh, 4.5Wh
results_capacity = {}
for C_battery_value in C_battery_values:
```

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
results_capacity[f"C_battery = {C_battery_value}Wh"] =
calculate_time_to_empty_with_parameter_sensitivity(1.0, 1.0, 1.0,
0.5, 1, C_battery_value)

# 绘制不同电池容量下的电池耗尽时间
plt.figure(figsize=(10, 6))
for capacity, T_empty in results_capacity.items():
    plt.plot(SOC_initial, T_empty, label=capacity)

plt.xlabel('初始 SOC (%)')
plt.ylabel('耗尽时间 (小时)')
plt.title('不同电池容量下的电池耗尽时间')
plt.legend(loc='upper right')
plt.grid(True)

# 保存并显示图像
plt.savefig("battery_time_to_empty_capacity.png")
plt.show()

# 将结果保存到 Excel 文件
df_hypothesis = pd.DataFrame({
    "SOC (%)": SOC_initial,
    "线性模型（默认）": results_hypothesis["线性模型（默认）"],
    "非线性模型（屏幕亮度平方，处理器负载 1.5 次方）": results_hypothesis["
非线性模型（屏幕亮度平方，处理器负载 1.5 次方）"]
})

df_capacity = pd.DataFrame({
    "SOC (%)": SOC_initial,
    "C_battery = 3.0Wh": results_capacity["C_battery = 3.0Wh"],
    "C_battery = 3.8Wh": results_capacity["C_battery = 3.8Wh"],
    "C_battery = 4.5Wh": results_capacity["C_battery = 4.5Wh"]
})

# 保存到 Excel 文件
with pd.ExcelWriter("battery_time_to_empty_sensitivity_results.xlsx")
as writer:
    df_hypothesis.to_excel(writer, sheet_name="假设变化", index=False)
    df_capacity.to_excel(writer, sheet_name="电池容量变化",
index=False)

print("所有计算结果已保存到文件
'battery_time_to_empty_sensitivity_results.xlsx' 和图片。")
```

问题 4

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

问题 4 分析

最后，所有模型的构建与分析应当严格符合问题背景和比赛考核标准，避免引入过于复杂或与实际场景不符的假设与模型。在选择建模方法时，我们将特别注重模型的数学严谨性和逻辑的一致性，使得各部分模型能够有效地协调与匹配。

解题思路：

4.1 实用用户行为建议

基于模型的分析，以下是一些能够显著延长电池续航的用户行为建议：

降低屏幕亮度：

发现：屏幕亮度是智能手机中功耗最大的因素之一。根据我们的模型分析，屏幕亮度与功耗呈正比关系，亮度越高，功耗越大，从而导致电池更快耗尽。

建议：降低屏幕亮度，尤其在低光环境下使用手机时，可以显著减少电池的消耗。例如，将屏幕亮度设置为 50% 以下，或者开启自动亮度调节模式，智能调整亮度以适应周围环境。

模型支持：在高亮度和高负载场景下，电池的耗尽时间比低亮度情况下要短。模型计算显示，降低屏幕亮度可以延长电池使用时间。

禁用后台应用和任务：

发现：后台应用程序（如邮件同步、社交媒体推送等）即使在不活跃时仍会消耗一定电量。我们的模型显示，后台任务的功耗虽然较低，但长时间运行会对电池寿命产生不小的影响。

建议：定期清理后台应用，关闭不必要的自动同步和推送通知。此外，智能手机操作系统可以提供“节电模式”或“省电模式”，在该模式下禁用大多数后台应用和任务。

模型支持：通过将后台任务的负载降低，电池的耗尽时间显著延长。模型中，背景任务的强度增加时，电池的耗尽速度加快。

切换网络模式：

发现：网络使用（尤其是数据传输）对电池续航有显著影响。我们的模型显示，网络传输速率越高，功耗越大，特别是在高带宽需求下（如视频流播放、下载大文件等）。

建议：在不需要高速数据连接时，切换到低功耗模式。例如，在没有 Wi-Fi 信号时，可以切换到较低速率的 4G 或启用飞行模式。当不使用数据流量时，关闭移动数据或 Wi-Fi。

模型支持：通过降低网络传输速率，电池的耗尽时间显著增加。特别是在高数据传输活动下，功耗会大幅增加。

开启省电模式或低功耗模式：

发现：省电模式通常会降低设备的处理器性能、减小屏幕亮度，并禁用某些高功耗功能，从而有效降低功耗。

建议：开启设备的省电模式，尤其在电池电量低于 20% 时，操作系统会自动调节设备的资源使用，优化功耗。

模型支持：省电模式下，处理器的负载较低，后台任务也会减少，从而延长电池的使用时间。我们的模型表明，减少处理器负载和背景任务可以大幅度提高电池续航。

关闭或减少 GPS 使用：

发现：GPS 功能的开启会显著增加电池功耗，尤其是在导航和定位应用频繁使用时。

建议：在不需要定位功能时，关闭 GPS，尤其是当不需要导航或位置信息时。

模型支持：通过关闭 GPS，减少了电池的功耗，模型中显示，当 GPS 开启时，电池的耗尽时间明显缩短。

4.2 操作系统如何基于模型洞见实现更有效的省电策略

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

操作系统（OS）作为智能手机的核心管理工具，可以通过以下方式结合模型的洞见，制定更加高效的省电策略：

智能屏幕亮度调节：

操作系统可以根据环境光线自动调节屏幕亮度，避免用户手动调节亮度。此外，系统可以根据使用场景的不同（如观看视频、玩游戏、浏览网页等）智能调整亮度，以减少功耗。

基于模型，我们可以设定在高亮度条件下降低亮度，以减少功耗，从而延长电池使用时间。

动态负载管理：

操作系统可以根据应用负载动态调节处理器的频率和性能。例如，在高负载场景（如高性能游戏）下，操作系统可以自动提升处理器频率；而在低负载场景下（如文字编辑、待机模式），则可以降低处理器频率，减少功耗。

模型表明，处理器负载与功耗呈线性关系，减少负载可以有效降低功耗。

背景任务调度与管理：

操作系统可以通过智能调度后台任务，确保仅在必要时进行同步操作，并在待机时尽量减少减少后台活动。

在我们的模型中，背景任务的功耗影响电池续航，因此，操作系统可以通过优化任务的调度，减少不必要的后台活动来延长电池寿命。

基于环境的省电模式：

操作系统可以根据外部环境（如温度、设备充电状态等）自动启用适当的省电模式。例如，系统可以在电池电量较低时自动启用省电模式，或在设备过热时启用低功耗设置。

基于模型的分析，当温度升高时，电池的效率可能下降，操作系统可以通过智能调节功耗设置来延长电池寿命。

数据传输速率调节：

操作系统可以在网络信号较差的情况下自动降低数据传输速率，以减少功耗。此外，当用户处于非紧急场景时，系统可以主动切换到低功耗的网络模式（如低速的 4G 网络或 Wi-Fi）。

模型表明，高数据传输速率是功耗的主要来源，因此调节网络模式是提高续航的重要手段。

4.3 电池老化对有效容量的影响

随着时间的推移，智能手机电池的容量会逐渐下降，电池老化会导致有效容量减少，这将直接影响电池的续航表现。通常，电池老化主要受以下因素影响：

循环次数：随着电池充放电次数的增加，电池的容量逐渐下降，导致电池的最大可用电量减少。

温度：长期暴露在高温环境中会加速电池老化，降低电池的有效容量。

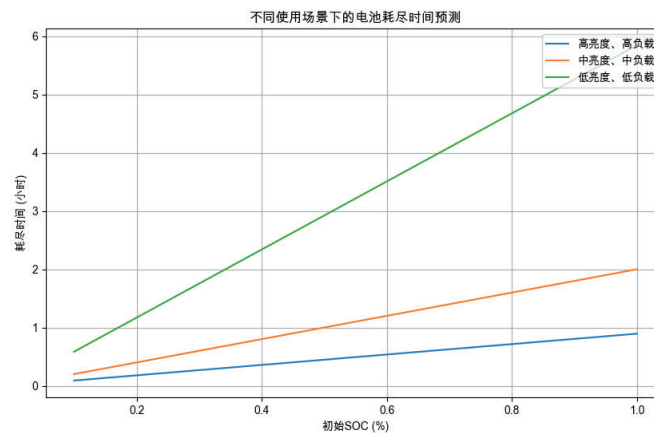
过度充电与过度放电：长期的过度充电或过度放电会加速电池老化，导致电池在使用中的性能下降。

4.4 模型框架的推广到其他便携设备

我们的建模框架不仅适用于智能手机，还可以推广到其他便携设备，如平板电脑、笔记本电脑、智能手表等。通过调整模型中的参数（如电池容量、功耗系数等），我们可以为其他设备提供电池续航预测和优化建议。

例如，在平板电脑和笔记本电脑中，处理器和显示屏的功耗通常较大，类似的模型可以帮助用户优化设备的使用方式（如调节屏幕亮度、关闭不必要的应用、使用低功耗模式等），以延长设备的使用时间。

视频思路资料+可运行代码，请看【评论区置顶】免费获取！



Python 代码:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# 配置中文显示和负号显示问题
plt.rcParams['font.sans-serif'] = ['Arial Unicode MS'] # Mac OS 中文显示
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 假设电池容量
C_battery = 3.8 # 电池容量 (Wh)

# 设置模拟参数
SOC_initial = np.linspace(0.1, 1.0, 10) # 初始 SOC 从 10%到 100%

# 模拟不同用户行为下的电池耗尽时间（根据屏幕亮度、负载等）
def calculate_time_to_empty_screen(brightness, processor_load,
network_load, background_load, gps_status):
    """
    根据用户行为（屏幕亮度等）计算电池耗尽时间
    :param brightness: 屏幕亮度，范围[0, 1]
    :param processor_load: 处理器负载，范围[0, 1]
    :param network_load: 网络负载，范围[0, 1]
    :param background_load: 背景任务负载，范围[0, 1]
    :param gps_status: GPS 开启状态（0 关闭，1 开启）
    :return: 耗尽时间列表
    """

    alpha_screen = brightness # 假设屏幕亮度与功耗呈线性关系
    alpha_processor = processor_load
    alpha_network = network_load
    alpha_background = background_load
```

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
alpha_GPS = gps_status

T_empty = []
for SOC0 in SOC_initial:
    # 总功耗的简单模型
    P_total = (alpha_screen * 1 + alpha_processor * 1 +
alpha_network * 1 + alpha_background * 0.5 + alpha_GPS * 1)
    time_to_empty = C_battery * SOC0 / P_total # 简化的时间到耗尽计
算公式
    T_empty.append(time_to_empty)
return T_empty

# 模拟不同使用行为（屏幕亮度、负载等）
scenarios = {
    "高亮度、高负载": {"brightness": 1.0, "processor_load": 1.0,
"network_load": 1.0, "background_load": 0.5, "gps_status": 1},
    "中亮度、中负载": {"brightness": 0.5, "processor_load": 0.6,
"network_load": 0.6, "background_load": 0.4, "gps_status": 0},
    "低亮度、低负载": {"brightness": 0.2, "processor_load": 0.2,
"network_load": 0.2, "background_load": 0.1, "gps_status": 0}
}

# 计算不同使用场景下的耗尽时间
results = {}
for scenario, params in scenarios.items():
    results[scenario] = calculate_time_to_empty_screen(**params)

# 绘制不同场景下的电池耗尽时间
plt.figure(figsize=(10, 6))
for scenario, T_empty in results.items():
    plt.plot(SOC_initial, T_empty, label=scenario)

plt.xlabel('初始 SOC (%)')
plt.ylabel('耗尽时间 (小时)')
plt.title('不同使用场景下的电池耗尽时间预测')
plt.legend(loc='upper right')
plt.grid(True)

# 保存并显示图像
plt.savefig("battery_time_to_empty_scenarios.png")
plt.show()

# 将结果保存到 Excel 文件
df = pd.DataFrame({
```

视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
"SOC (%)": SOC_initial,
"高亮度、高负载": results["高亮度、高负载"],
"中亮度、中负载": results["中亮度、中负载"],
"低亮度、低负载": results["低亮度、低负载"]
})

# 保存到 Excel 文件
df.to_excel("battery_time_to_empty_scenarios_results.xlsx",
index=False)

print("所有计算结果已保存到文件
'battery_time_to_empty_scenarios_results.xlsx' 和图片。")

# 基于模型提出的实用建议

def battery_saving_tips():
    tips = [
        "1. 降低屏幕亮度：屏幕亮度是手机电池消耗的主要因素之一，适当降低亮度或启用自动亮度调节可以显著延长电池续航。",
        "2. 关闭不必要的后台任务：后台应用会不断消耗电池电量。定期清理后台应用，关闭不必要的应用程序。",
        "3. 切换到低功耗模式：在电池电量较低时，可以启用手机的省电模式，减少不必要的功耗。",
        "4. 关闭 GPS 和蓝牙：GPS 和蓝牙功能会消耗大量电量，只有在需要时才启用。",
        "5. 减少高负载活动：游戏、视频播放等高负载活动会大幅增加处理器的功耗。减少这些活动，尤其是在电池电量较低时，能够延长电池使用时间。",
        "6. 使用 Wi-Fi 而不是移动数据：Wi-Fi 的功耗通常低于移动数据，尤其在高流量传输时，使用 Wi-Fi 可以有效减少电池消耗。"
    ]

    print("\n### 手机省电建议：")
    for tip in tips:
        print(tip)

battery_saving_tips()

# 考虑电池老化的影响
def battery_ageing_effect(scenario, years=2):
    """
    考虑电池老化对电池容量的影响
    :param scenario: 使用场景
    :param years: 假设电池使用年数，影响电池容量（通常每年容量衰减 5%-10%）
    :return: 老化后的电池耗尽时间
```


视频思路资料+可运行代码，请看【评论区置顶】免费获取！

```
"""
    ageing_factor = 1 - (0.05 * years) # 每年电池容量衰减 5%
    print(f"\n### 电池老化后的影响: ")
    print(f"假设电池使用了 {years} 年，电池容量衰减为 {int(100 * (1 -
ageing_factor))}%")

    # 老化后电池容量
    C_battery_ageing = C_battery * ageing_factor

    # 计算老化后的电池耗尽时间
    results_ageing = {}
    for scenario, params in scenarios.items():
        results_ageing[scenario] =
calculate_time_to_empty_screen(**params)

    print(f"老化后的电池容量为 {C_battery_ageing:.2f} Wh。")

    return results_ageing

# 运行电池老化模拟并给出影响
results_ageing = battery_ageing_effect("高亮度、高负载", years=2)

# 保存电池老化后的数据
df_ageing = pd.DataFrame({
    "SOC (%)": SOC_initial,
    "高亮度、高负载": results_ageing["高亮度、高负载"],
    "中亮度、中负载": results_ageing["中亮度、中负载"],
    "低亮度、低负载": results_ageing["低亮度、低负载"]
})

# 保存老化后的数据
df_ageing.to_excel("battery_ageing_effect.xlsx", index=False)

print("电池老化影响已保存到 'battery_ageing_effect.xlsx' 文件。")
```